

UNIVERSIDADE CATÓLICA DO SALVADOR — UCSAL

Programa de Iniciação Científica — PROVIC

Sistemas de Análise e Desenvolvimento

SGF — Sistema de Gestão de Fiscalização

Relatório Técnico de Desenvolvimento de Software

Orientador

Prof. Rafael Bispo

Namespace do projeto:

Provic Ucsal

Modalidade Iniciação científica

Salvador, Bahia — 2025

1. Introdução

O presente relatório documenta o desenvolvimento do SGF — Sistema de Gestão de Fiscalização, um software fullstack desenvolvido no âmbito do Programa de Iniciação Científica (PROVIC) da Universidade Católica do Salvador (UCSAL). O sistema foi concebido para apoiar operações de fiscalização de obras públicas municipais, oferecendo controle de cadastro, acompanhamento de status, registro de vistorias e geração de relatórios.

O trabalho envolveu o ciclo completo de desenvolvimento de software: análise de requisitos, modelagem de dados, implementação do backend com API REST, desenvolvimento do frontend como Single Page Application (SPA), aplicação de práticas de segurança, auditoria técnica e iterações de melhoria baseadas nos pontos identificados na revisão.

O projeto é organizado como monorepo, com duas aplicações independentes que se comunicam por API REST autenticada: o backend em Java 21 com Spring Boot e o dashboard em React 19 com Vite. Ao longo do desenvolvimento, o sistema evoluiu por seis versões de backend (v1.0 a v6.0) e dez versões de frontend (v1 a v10), cada uma incorporando novos requisitos e correções identificadas em revisões técnicas.

2. Objetivos

2.1 Objetivo Geral

Desenvolver uma plataforma fullstack completa para gestão e fiscalização de obras públicas municipais, aplicando boas práticas de engenharia de software, arquitetura em camadas, segurança por design e conformidade com normas de proteção de dados.

2.2 Objetivos Específicos

- Modelar o domínio de fiscalização de obras públicas com entidades JPA, relacionamentos e constraints de integridade no banco de dados.
- Implementar API REST completa com 34 endpoints organizados por domínio, cobrindo operações CRUD e fluxos de negócio específicos.
- Aplicar autenticação stateless com JWT e controle de acesso baseado em papéis (RBAC) com três níveis de permissão distintos.
- Desenvolver interface de usuário responsiva com React 19, tema claro/escuro, navegação por rotas protegidas e exportação de dados.
- Implementar sistema de auditoria de ações para rastreabilidade conforme requisitos da LGPD (Lei 13.709/2018, Art. 37).
- Realizar auditoria técnica independente do sistema e corrigir todas as vulnerabilidades e problemas de qualidade identificados.
- Produzir documentação técnica completa: README de backend, README de frontend, README de monorepo e relatório técnico.

3. Metodologia

O desenvolvimento seguiu uma abordagem iterativa e incremental, próxima ao modelo ágil, com ciclos curtos de implementação, teste manual e revisão. Cada versão do sistema incorporou funcionalidades completas e testáveis, evitando entregas parciais que impedissem a avaliação do funcionamento integrado.

3.1 Processo de Desenvolvimento

O ciclo de desenvolvimento de cada funcionalidade seguiu a sequência: (1) análise do requisito e modelagem da solução, (2) implementação da migration de banco de dados, (3) criação das entidades JPA e repositórios, (4) implementação do service com regras de negócio, (5) exposição via controller REST, (6) atualização do frontend para consumo do endpoint e (7) teste de integração manual.

3.2 Controle de Versão

O versionamento do banco de dados foi gerenciado com Flyway, garantindo que cada mudança de schema fosse rastreável, reproduzível e aplicada de forma idempotente. O código-fonte foi organizado em monorepo com as aplicações backend e frontend como módulos independentes, facilitando o versionamento conjunto e a revisão cruzada de mudanças.

3.3 Auditoria Técnica

Ao atingir a versão 5.0 do backend, foi realizada uma auditoria técnica formal baseada no modelo de revisão de sistemas governamentais em produção. A auditoria avaliou dez dimensões: arquitetura geral, modelagem de domínio, design de API, segurança, banco de dados, regras de negócio, qualidade de código, escalabilidade, observabilidade e riscos críticos. Todos os problemas identificados foram corrigidos na versão 6.0.

4. Arquitetura do Sistema

4.1 Visão Geral

O SGF adota arquitetura cliente-servidor com separação clara entre as camadas de apresentação (frontend SPA) e de lógica de negócio (backend API REST). A comunicação entre as camadas ocorre exclusivamente via HTTP com autenticação JWT, sem acoplamento de código entre os projetos.

Camada	Tecnologia e Versão
Frontend (SPA)	React 19 + Vite 8 + Tailwind CSS 3.4
Roteamento	React Router DOM 7.14
HTTP Client	Axios 1.15 com interceptors JWT

Gráficos	Recharts 3.8
Backend (API)	Java 21 + Spring Boot 3.4.5
Segurança	Spring Security 6 + JWT 0.12.6
Persistência	Spring Data JPA + Hibernate
Banco de dados	PostgreSQL 16
Migrações	Flyway 10 (6 versões)
Build backend	Maven 3.9
Build frontend	npm 10 + Vite

4.2 Arquitetura do Backend

O backend segue a arquitetura em camadas clássica de sistemas Spring Boot, com responsabilidades claramente delimitadas entre os pacotes:

Pacote	Responsabilidade
controller/	Recebe requisições HTTP, valida com @Valid, delega ao service
service/	Contém regras de negócio, orquestração e transações
repository/	Abstração JPA de acesso a dados (Spring Data)
model/	Entidades JPA mapeadas ao banco de dados
dto/request/	Payloads de entrada validados com Jakarta Bean Validation
dto/response/	Payloads de saída com controle explícito do que é serializado
security/	Filtro JWT, configuração Spring Security e rate limiting
exception/	Handler global de exceções com respostas padronizadas

4.3 Arquitetura do Frontend

O dashboard é uma SPA (Single Page Application) com roteamento client-side via React Router. O estado de autenticação é gerenciado globalmente pelo AuthContext e o tema claro/escuro pelo ThemeContext. A comunicação com a API é centralizada em dois arquivos: client.js (instância Axios com interceptors) e sgf.js (funções de chamada organizadas por domínio).

5. Modelagem de Dados

5.1 Entidades e Relacionamentos

O modelo de dados é composto por sete tabelas principais, relacionadas para representar o ciclo completo de fiscalização de obras. As chaves primárias são do tipo UUID gerado pelo PostgreSQL via `gen_random_uuid()`, e todas as colunas de enumeração possuem constraints CHECK no DDL.

Tabela	Descrição	Relacionamentos
usuario	Usuários do sistema com papel e hash de senha BCrypt	1:N vistoria
obra	Obras fiscalizadas com código único e status controlado	1:N item, 1:N vistoria
item	Itens de uma obra classificados por criticidade	N:1 obra, N:1 classificacao
classificacao	Lookup de classes A, B, C com critério descritivo	1:N item
vistoria	Registro de uma visita de fiscalização a uma obra	N:1 obra, N:1 usuario, 1:N log_fiscalizacao
log_fiscalizacao	Resultado individual de um item em uma vistoria	N:1 vistoria, N:1 item, N:1 origem_dado
origem_dado	Lookup do tipo de evidência usada no log	1:N log_fiscalizacao
audit_log	Registro imutável de ações para rastreabilidade (LGPD)	N:1 usuario (nullable)

5.2 Fluxos de Status

As transições de status são controladas por máquina de estados implementada no `ObraService`, impedindo que qualquer transição inválida seja executada, independentemente do papel do usuário. O status inicial de uma obra é sempre `PLANEJADA`, nunca recebido do cliente.

Fluxo	Transições Permitidas
Obra: PLANEJADA	→ EM_ANDAMENTO
Obra: EM_ANDAMENTO	→ PARALISADA ou → CONCLUIDA
Obra: PARALISADA	→ EM_ANDAMENTO
Obra: CONCLUIDA	Estado terminal — sem transições
Item: PENDENTE	→ EM_VISTORIA
Item: EM_VISTORIA	→ APROVADO ou → REPROVADO

5.3 Migrações Flyway

Todas as mudanças de schema são versionadas com Flyway, garantindo rastreabilidade e aplicação determinística em qualquer ambiente.

Versão	Arquivo	Conteúdo
V1	create_tables.sql	DDL completo: tabelas, PKs, FKs, constraints CHECK
V2	seed_data.sql	Dados iniciais: classificações, origens, usuários e obra de exemplo
V3	add_indexes.sql	Índices de performance: status, GIN full-text, compostos por obra
V4	fix_vistoria_nullable.sql	usuario_id nullable na vistoria; FK com ON DELETE SET NULL
V5	add_auth.sql	Coluna senha_hash (BCrypt) na tabela usuario
V6	audit_log.sql	Coluna atualizado_em nas entidades mutáveis + tabela audit_log + triggers

6. API REST

6.1 Visão Geral dos Endpoints

A API expõe 34 endpoints organizados por domínio. Cada endpoint segue o padrão REST: uso correto dos verbos HTTP, retornos semânticos (200, 201, 204, 400, 401, 403, 404, 409, 422, 429) e respostas de erro padronizadas pelo GlobalExceptionHandler.

Domínio	Endpoints	Métodos	Acesso mínimo
Auth	/auth/login	POST	Público
Auth	/auth/definir-senha/{id}	POST	ADMIN
Auth	/auth/alterar-minha-senha	POST	Autenticado
Auth	/auth/me	GET	Autenticado
Obras	/obras	GET, POST	FISCAL / GESTOR
Obras	/obras/buscar	GET	FISCAL
Obras	/obras/{id}	GET	FISCAL
Obras	/obras/{id}/dashboard	GET	FISCAL
Obras	/obras/{id}/timeline	GET	FISCAL
Obras	/obras/{id}	PUT	GESTOR
Obras	/obras/{id}/status	PATCH	FISCAL
Obras	/obras/{id}	DELETE	ADMIN

Itens	/itens, /itens/buscar	GET	FISCAL
Itens	/itens/{id}	GET, POST	FISCAL
Itens	/itens/{id}	PUT	GESTOR
Itens	/itens/{id}/status	PATCH	FISCAL
Itens	/itens/{id}	DELETE	GESTOR
Vistorias	/vistorias	GET, POST	FISCAL
Vistorias	/vistorias/{id}	GET	FISCAL
Logs	/logs	GET, POST	FISCAL
Usuários	/usuarios	GET, POST	FISCAL / ADMIN
Usuários	/usuarios/{id}	GET, PUT, DELETE	FISCAL / ADMIN
Usuários	/usuarios/{id}/papel	PATCH	ADMIN
Lookups	/classificacoes, /origens-dado	GET	Público

7. Segurança

7.1 Estratégia de Autenticação

O sistema utiliza autenticação stateless com JSON Web Token (JWT), eliminando a necessidade de armazenamento de sessão no servidor. Cada token gerado contém as claims: subject (email), papel, jti (identificador único do token), iat (emissão) e exp (expiração de 24 horas). A assinatura utiliza algoritmo HMAC-SHA256 com segredo de no mínimo 32 caracteres, configurado via variável de ambiente.

7.2 Controle de Acesso por Papel (RBAC)

Três papéis com permissões progressivas são suportados: FISCAL (visualiza e registra), GESTOR (cria e edita obras e itens) e ADMIN (acesso total incluindo gerenciamento de usuários). As restrições são aplicadas em duas camadas: SecurityConfig (por padrão de URL e método HTTP) e @PreAuthorize nos métodos de service (defesa em profundidade).

Permissão	FISCAL	GESTOR	ADMIN
Ver obras, itens, vistorias	Sim	Sim	Sim
Registrar vistoria e log	Sim	Sim	Sim
Alterar status (obra/item)	Sim	Sim	Sim
Criar e editar obras	Não	Sim	Sim
Editar e deletar itens	Não	Sim	Sim
Deletar obras	Não	Não	Sim
Gerenciar usuários (CRUD)	Não	Não	Sim
Definir senha de usuários	Não	Não	Sim

7.3 Proteções Implementadas

- Rate limiting no endpoint de login: máximo 10 tentativas por IP em 60 segundos (sliding window). Retorna HTTP 429 com header Retry-After.
- BCrypt com fator de custo 12 para hash de senhas, garantindo aproximadamente 250ms por operação de verificação.
- Papel do usuário relido do banco em cada requisição pelo JwtAuthFilter, eliminando o delay de até 24h que existia quando papéis eram alterados durante uma sessão ativa.
- CORS restrito a origens configuradas via variável de ambiente CORS_ALLOWED_ORIGINS, eliminando a configuração anterior que aceitava qualquer origem.
- Emails mascarados nos logs: formato a**@dominio.com em vez de texto claro, em conformidade com a LGPD.

- Endpoint POST /auth/definir-senha/{id} protegido por @PreAuthorize('hasRole(ADMIN)'), corrigindo vulnerabilidade crítica onde qualquer pessoa podia redefinir qualquer senha.

8. Auditoria e Conformidade com LGPD

8.1 Sistema de Auditoria

A LGPD (Lei 13.709/2018, Art. 37) determina que controladores de dados pessoais devem manter registros das operações de tratamento de dados. Para atender a este requisito, foi implementado um sistema de auditoria baseado na entidade AuditLog, que registra de forma imutável toda ação relevante realizada no sistema.

Campo	Descrição
id	UUID único do registro — imutável
usuario_id	UUID do usuário que realizou a ação (nullable para sistema)
acao	Tipo: LOGIN, CREATE, UPDATE, DELETE, PATCH, ACESSO_NEGADO
entidade	Nome da entidade afetada: Obra, Usuario, Vistoria...
entidade_id	UUID da entidade afetada
descricao	Texto legível descrevendo a ação realizada
ip_origem	IP da requisição (IPv4 ou IPv6), considerando proxies reversos
criado_em	Timestamp imutável de criação do registro

8.2 Design do AuditService

O AuditService utiliza duas anotações Spring para garantir confiabilidade sem impactar a performance: @Async para gravação assíncrona (não bloqueia a resposta ao cliente) e Propagation.REQUIRES_NEW para execução em transação separada, garantindo que o log seja persistido mesmo quando a transação principal sofre rollback — por exemplo, um login falhado ainda é registrado.

O serviço nunca lança exceções para o chamador: falhas na gravação do log são registradas no Logger mas não interrompem o fluxo principal da aplicação, seguindo o princípio de que a auditoria não deve ser um ponto de falha do sistema.

9. Frontend — SGF Dashboard

9.1 Estrutura e Navegação

O dashboard é composto por sete páginas acessadas via roteamento client-side. A autenticação é verificada pelo AuthContext na inicialização: se houver token JWT no localStorage, os dados do usuário são restaurados via GET /auth/me; caso contrário, a tela de Login é exibida. O interceptor Axios no client.js trata respostas 401 automaticamente, limpando a sessão e redirecionando para o login.

Página / Rota	Funcionalidade Principal	Acesso
Login /	Autenticação com email e senha, toggle de senha, feedback de erro	Público
Obras /	Listagem com filtros de status e busca em tempo real	Autenticado
Dashboard /obras/:id	KPIs, abas de itens/vistorias/timeline, exportação CSV	Autenticado
Vistorias /relatorios	Histórico global de vistorias com filtro por obra	Autenticado
Analytics /analytics	Gráficos agregados com Recharts	GESTOR+
Usuários /usuarios	CRUD com modal de remoção forçada	ADMIN
Configurações /configuracoes	Tema, animações, notificações — persistidos em localStorage	Autenticado

9.2 Sistema de Temas

O tema claro/escuro é implementado via CSS variables aplicadas ao atributo data-theme do elemento HTML raiz, sem lógica condicional nos componentes. A preferência é detectada automaticamente via prefers-color-scheme na inicialização e persistida em localStorage. A alternância é transparente para o JSX de todos os componentes, que apenas utilizam variáveis como var(--card) e var(--text).

9.3 Exportação de CSV

A exportação de dados de uma obra para CSV é gerada client-side via Blob, sem chamada ao backend. O arquivo inclui: resumo da obra, KPIs calculados, tabela de itens e histórico de vistorias. Um BOM UTF-8 (Byte Order Mark) é adicionado ao início do arquivo para compatibilidade com Microsoft Excel em sistemas Windows. O nome do arquivo segue o padrão `sgf_{codigo}_{data}.csv`.

10. Métricas do Projeto

10.1 Volume de Código

Módulo / Camada	Arquivos	Linhas de Código
Backend — controller/	8	550
Backend — service/	7	1.083
Backend — repository/	8	252
Backend — model/	11	459
Backend — dto/	21	374
Backend — security/	4	450
Backend — exception/	5	186
Backend — Migrações SQL	6	347
Frontend — pages/	7	2.879
Frontend — components/	6	1.249
Frontend — api/	2	230
Frontend — context/	2	159
Total geral	87	8.218

10.2 Cobertura de Funcionalidades

Funcionalidade	Status
API REST com 34 endpoints	Implementado
Autenticação JWT stateless	Implementado
RBAC com 3 papéis (ADMIN, GESTOR, FISCAL)	Implementado
Máquina de estados para obras e itens	Implementado
Rate limiting no login (10 req/60s por IP)	Implementado
Sistema de auditoria com AuditLog (LGPD)	Implementado
Migrações versionadas Flyway (V1 a V6)	Implementado
Dashboard com KPIs calculados via SQL	Implementado
Exportação de CSV client-side	Implementado
Tema claro/escuro com persistência	Implementado
Preferências de usuário persistidas	Implementado
Fiscal desvinculado tratado de forma amigável	Implementado
CORS restrito por variável de ambiente	Implementado

@PreAuthorize como defesa em profundidade	Implementado
---	--------------

11. Auditoria Técnica e Correções

Na versão 5.0 do backend, foi realizada uma auditoria técnica formal seguindo o modelo de avaliação de sistemas governamentais em produção. A auditoria avaliou dez dimensões de qualidade e identificou problemas classificados por gravidade. Todas as correções foram implementadas na versão 6.0.

11.1 Problemas Identificados e Corrigidos

Problema	Gravidade	Versão Corrigida	Solução Aplicada
POST /auth/definir-senha público — qualquer pessoa podia redefinir qualquer senha	Crítica	v6.0	@PreAuthorize('hasRole(ADMIN)') no controller e service
Papel do usuário vinha do token — delay de até 24h em alterações	Crítica	v6.0	JwtAuthFilter relê papel do banco em cada requisição
CORS totalmente aberto com allowedOriginPatterns('*')	Alta	v6.0	Restrito via variável de ambiente CORS_ALLOWED_ORIGINS
Sem rate limiting no login — vulnerável a brute force	Alta	v6.0	RateLimitFilter: sliding window 10 req/60s por IP
Status da obra vinha do DTO — bypassava máquina de estados	Alta	v6.0	ObraService sempre inicia com PLANEJADA; campo removido do DTO
Emails em texto claro nos logs — problema de conformidade LGPD	Média	v6.0	maskEmail() mascara para a**@dominio.com nos logs
FK verificada por nome de constraint (string frágil)	Média	v6.0	GlobalExceptionHandler usa SQLState PostgreSQL (23503, 23505)
Dashboard calculava contagens em memória via stream Java	Média	v6.0	ItemRepository com countByObraIdAndStatus() via SQL
Sem coluna atualizado_em nas tabelas mutáveis	Alta	v6.0	Migration V6 + @UpdateTimestamp + triggers PostgreSQL
Sem tabela de auditoria — ausência de rastreabilidade	Alta	v6.0	Entidade AuditLog + AuditService @Async + migration V6
Sem @PreAuthorize nos métodos — defesa em camada única	Média	v6.0	@PreAuthorize adicionado em todos os métodos sensíveis
CorsConfig.java criava configuração duplicada de CORS	Baixa	v6.0	Classe neutralizada — CORS centralizado no SecurityConfig

11.2 Resultado da Auditoria

Após a implementação de todas as correções na versão 6.0, o risco geral do sistema foi reclassificado de ALTO para BAIXO para ambiente de intranet municipal. Os problemas críticos de segurança foram eliminados, o sistema de auditoria foi implementado em conformidade com a LGPD, e mecanismos de defesa em profundidade foram adicionados.

12. Decisões Técnicas Relevantes

12.1 JWT Stateless

A escolha por autenticação stateless com JWT elimina a necessidade de armazenamento de sessão no servidor, facilitando escalabilidade horizontal. Em um sistema de fiscalização pública com múltiplos fiscais em campo, a ausência de estado no servidor é um benefício arquitetural direto. O jti (JWT ID) foi adicionado como base para suporte futuro a revogação individual de tokens.

12.2 Flyway em vez de ddl-auto: update

O Spring Boot oferece a opção `ddl-auto: update` que atualiza o schema automaticamente, mas esta abordagem é não-determinística em produção — pode deixar colunas obsoletas, não remove constraints e não possui mecanismo de rollback. Flyway garante rastreabilidade completa, versionamento explícito e execução idempotente de cada mudança, obrigatório para sistemas com dados reais.

12.3 ON DELETE SET NULL em `vistoria.usuario_id`

Quando um fiscal é removido do sistema, o histórico de vistorias que realizou não deve desaparecer — isso comprometeria a integridade do histórico de fiscalização, que é um documento público. A escolha de `ON DELETE SET NULL` preserva todos os registros de vistoria enquanto remove apenas o vínculo com o usuário. O frontend exibe o estado de forma amigável como 'Fiscal desvinculado' com badge visual, sem textos com colchetes ou mensagens técnicas para o usuário.

12.4 CSS Variables para Temas no Frontend

O tema claro/escuro é controlado por variáveis CSS aplicadas ao elemento raiz HTML, sem lógica condicional no JSX dos componentes. Qualquer componente usa `var(--card)`, `var(--text)`, etc., e a alternância de tema é completamente transparente para o código dos componentes. Isso evita a necessidade de classes Tailwind condicionais e mantém o código de UI desacoplado da lógica de tema.

12.5 CSV Gerado Client-Side

A exportação de dados de obras para CSV é gerada no browser via Blob, sem chamar o backend. Para o volume atual (uma obra por vez, dezenas de itens e vistorias), a

geração em memória no browser é instantânea e não adiciona carga ao servidor. Mover para o backend apenas faria sentido em cenários de relatórios multi-obra, geração de PDF ou dados que o frontend não possui acesso direto.

12.6 AuditService com @Async e REQUIRES_NEW

A gravação de audit logs é assíncrona (@Async) para não bloquear a resposta ao cliente, e usa Propagation.REQUIRES_NEW para executar em transação separada. Isso garante duas propriedades importantes: (1) o log é persistido mesmo quando a transação principal sofre rollback (ex: login com credenciais erradas ainda gera um registro de ACESSO_NEGADO), e (2) uma falha na gravação do log nunca interrompe o fluxo principal da aplicação.

13. Dificuldades Encontradas e Soluções

Dificuldade	Solução Adotada
FK com ON DELETE RESTRICT impedia remoção de usuário com vistorias mesmo com forcar=true	Migration V4: DROP NOT NULL em usuario_id + FK com ON DELETE SET NULL
Exportação CSV retornava status undefined por acesso a state fora do escopo do componente	Substituição de statusAtual (state) por dash.status (dado do objeto dash disponível na função)
JSX com painel de detalhe de item inválido por dois elementos irmãos sem container	Envolver o card de itens e o painel de detalhe em um React Fragment (<>...</>)
CORS duplo (WebMvcConfigurer + SecurityConfig) causava conflito de headers	Centralização exclusiva no CorsConfigurationSource do SecurityConfig
GlobalExceptionHandler verificava constraint por nome string — frágil a refatorações	Substituição por verificação de SQLState PostgreSQL (23503 = FK violation)
Papel do usuário no JwtAuthFilter vinha do token — não refletia alterações imediatas	findByEmail() no JwtAuthFilter para releitura do papel atual do banco
Dashboard calculando KPIs com stream Java em memória — ineficiente para obras grandes	Queries de contagem SQL via ItemRepository.countByObraIdAndStatus()

14. Conclusão

O SGF — Sistema de Gestão de Fiscalização foi desenvolvido e entregue como uma aplicação fullstack funcional, cobrindo o ciclo completo de fiscalização de obras públicas. O sistema passou por seis versões de backend e dez versões de frontend, incorporando progressivamente novos requisitos, correções de bugs, melhorias de segurança e conformidade com boas práticas de engenharia de software.

O ponto mais significativo do processo de desenvolvimento foi a auditoria técnica realizada na versão 5.0, que identificou doze problemas — três classificados como críticos. A correção sistemática de todos esses problemas na versão 6.0 elevou a qualidade do sistema a um nível adequado para ambiente de intranet municipal, com as seguintes garantias: autenticação JWT com papel sempre sincronizado com o banco, proteção contra brute force no login, CORS restrito, auditoria de ações conforme LGPD e defesa em profundidade via `@PreAuthorize`.

Do ponto de vista técnico, o projeto consolidou competências em Java 21, Spring Boot 3, Spring Security 6, JPA/Hibernate, PostgreSQL, Flyway, React 19, Vite, Tailwind CSS, JWT e autenticação stateless. Do ponto de vista acadêmico, o processo de auditoria e correção foi a experiência de aprendizado mais rica, pois exigiu raciocínio crítico sobre decisões arquiteturais, implicações de segurança e trade-offs de design.

O sistema está documentado com três READMEs (backend, frontend e monorepo) e este relatório técnico, formando um corpus de documentação completo para avaliação acadêmica e apresentação como portfólio profissional.

15. Referências

- BRASIL. Lei n. 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, 2018.
- SPRING. Spring Boot Reference Documentation, versão 3.4.5. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: mai. 2025.
- SPRING. Spring Security Reference Documentation. Disponível em: <https://docs.spring.io/spring-security/>. Acesso em: mai. 2025.
- JWT.IO. JSON Web Tokens — Introduction. Disponível em: <https://jwt.io/introduction>. Acesso em: mai. 2025.
- REACT. React Documentation (v19). Disponível em: <https://react.dev/>. Acesso em: mai. 2025.
- FLYWAY. Flyway Documentation — Migrations. Disponível em: <https://documentation.red-gate.com/flyway/>. Acesso em: mai. 2025.
- OWASP. OWASP Top Ten. Disponível em: <https://owasp.org/www-project-top-ten/>. Acesso em: mai. 2025.
- POSTGRESQL. PostgreSQL 16 Documentation. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: mai. 2025.